Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

2026
Vladimír Jančár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026
Vladimír Jančár

16681876

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Vladimír Jančár
Study programme:	Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Machine learning for generation of graph of given degree and girth.

Annotation: Generating graphs with prescribed structural properties is a key task in combinatorial optimization, network design, and bioinformatics. Traditional algorithmic approaches (e.g., configuration model) often fail to find graphs with high girth and specific degree because the space of possible solutions is vast and the constraints are nontrivial.

Aim: The goal is to investigate potential applications of machine learning to problems in algebraic graph theory. The student will have the opportunity to engage with the state-of-art research in machine learning and algebraic graph theory and contribute to the field by generating new record graphs and training neural networks to predict algebraic properties.

Literature: Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., & Grohe, M. (2019). Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks. _Proceedings of the AAAI Conference on Artificial Intelligence_, _33_(01), 4602–4609. [https://doi.org/10/ggfn97] (https://doi.org/10/ggfn97)
Paolino, R., Maskey, S., Welke, P., & Kutyniok, G. (2024). _Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning_.
Wu, L., Cui, P., Pei, J., & Zhao, L. (Eds.). (2022). _Graph Neural Networks: Foundations, Frontiers, and Applications_. Springer Nature Singapore. [https://doi.org/10.1007/978-981-16-6054-2] (https://doi.org/10.1007/978-981-16-6054-2)

Supervisor:	Mgr. Ján Pastorek
Department:	FMFI.KAI - Department of Applied Informatics
Head of department:	doc. RNDr. Tatiana Jajcayová, PhD.
Assigned:	25.02.2025

Approved:

Guarantor of Study Programme

.....
Student

.....
Supervisor

Acknowledgments:

[Acknowledgments to be written near the end of the thesis. Mention the supervisor, people who helped with experiments or computing resources, and any personal thanks that should appear in the submitted version.]

1 Abstract

This thesis investigates machine-learning methods for generating graphs of prescribed degree and girth, with emphasis on graph neural networks and constrained search for (k, g)-graphs.

[Final abstract to be written after the generator-comparison results are fixed. Briefly state the problem, the progression from prediction tasks to direct RL and voltage lifts, the main experimental result, the negative result if ML does not beat structured baselines, and the final interpretation.]

Keywords: graph neural networks, algebraic graph theory, cage problem, (k, g)-graphs, voltage graph lifts, heuristic search

Contents

List of Figures

List of Tables

2 Introduction

This thesis concerns machine-learning methods for generating graphs of prescribed degree and girth. In graph-theoretic language, the central objects are (k, g)-graphs: graphs in which every vertex has degree k and every cycle has length at least g. The degree fixes the local shape of the graph, while the girth says that short loops are forbidden. The classical cage problem asks for such graphs with the smallest possible number of vertices [1].

Graphs with controlled degree and large girth appear outside pure graph theory. In error-correcting codes, sparse graphs describe which symbols check each other. Short cycles make the decoding messages depend on each other too early, while larger girth keeps the local structure tree-like for more decoding steps [2]. In communication networks and supercomputer interconnects, regular high-girth or near-Moore graphs give many well-spread connections with few short loops, which can reduce congestion and latency [3].

The mathematical problem is therefore simple to state but difficult to solve: for fixed k and g, construct a small graph satisfying both constraints. Known approaches use algebraic constructions, voltage graph lifts, exhaustive search, heuristic search, and local modification techniques [4]. This thesis asks whether graph neural networks can help in this setting. The expected role of machine learning is not to replace exact graph-theoretic checks, but to guide a search toward promising choices.

The thesis is organized around the following research questions:

- Which graph neural network architectures can learn the local and cycle-related properties that appear in the definition of a (k, g)-graph?
- Can reinforcement learning construct small (k, g)-graphs by editing edges?
- Does an algebraic representation, in particular voltage graph lifts, give a more useful search space?
- Does learning improve this constrained search, or do non-learned methods remain stronger?

The progression of the thesis follows these questions. First, degree prediction and minimum-cycle prediction test whether the models can learn graph properties related to the target constraints. Then the construction problem is treated as a sequential decision problem and trained with reinforcement learning. Finally, the work moves to voltage graph lifts, where regularity is built into the construction and the search focuses on voltage assignments.

Chapters 2 and 3 introduce the needed notation and related construction methods. Chapter 4 reports the preparatory GNN tasks. Chapter 5 explains why the project moved from supervised guidance to reinforcement learning and then to constrained algebraic search. Chapter 6 describes voltage graph lifts. Chapter 7 reports the

current experimental results, and Chapter 8 discusses the most concrete future directions.

3 Background and Definitions

This chapter fixes the notation used in the rest of the thesis. Only the definitions needed for the experiments are stated here. More specialized constructions, such as voltage graph lifts, are introduced later at the point where they become part of the method.

3.1 Graphs

A graph is a pair $G = (V, E)$, where V is a finite set of vertices and E is a set of edges. In the main construction setting, graphs are simple and undirected unless stated otherwise. For a vertex $v \in V$, the degree of v, denoted $\deg(v)$, is the number of edges incident with v. The open neighborhood of v, denoted $N(v)$, is the set of vertices adjacent to v.

A graph is k-regular if every vertex has degree k. A walk is a sequence of vertices in which consecutive vertices are adjacent. A path is a walk with no repeated vertices. A cycle is a closed path of length at least 3. The girth of a graph, denoted $\operatorname{girth}(G)$, is the length of its shortest cycle. If a graph has no cycles, its girth is treated as infinite.

3.2 (k, g)-Graphs and Cages

A (k, g)-graph is a k-regular graph whose girth is at least g. A (k, g)-cage is a smallest k-regular graph of girth g; its order is commonly denoted $n(k, g)$. In search algorithms it is convenient to test the condition $\operatorname{girth}(G) \geq g$, because this accepts every valid candidate for a target lower bound on girth. When comparing with known graph tables, however, the exact girth and the number of vertices must still be reported.

The Moore bound gives a general lower bound on the order of a (k, g)-graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

For $k \geq 2$ and $g \geq 3$, the Moore bound is

$$M(k, g) = \begin{cases} 1 + k \sum_{i=0}^{\frac{g-3}{2}} (k-1)^i \\ \text{if } \ g \ \text{ is odd} \\ 2 \sum_{i=0}^{\frac{g-2}{2}} (k-1)^i \\ \text{if } \ g \ \text{ is even} \end{cases}.$$

A graph meeting this bound is called a Moore graph. Moore graphs are rare. For most parameter pairs, the practical problem is therefore to close the gap between the Moore lower bound and the smallest graph currently known.

3.3 Graph Neural Networks

Ordinary feed-forward neural networks expect fixed-size vectors. Graphs do not have a fixed number of vertices, and their vertices do not have a canonical ordering. Graph neural networks address this by applying the same local update rule at every vertex. This makes the model independent of the input graph size and equivariant to relabeling of vertices.

Message-passing graph neural networks process graph-structured data by iteratively updating vertex representations through layers [5]. At layer t, each vertex v has a hidden representation h_v^t. The layer aggregates messages from the neighborhood $N(v)$ and then updates the representation:

$$m_v^t = \mathrm{AGG}(\{h_u^{t-1} : u \in N(v)\})$$

$$h_v^t = \mathrm{UPD}(h_v^{t-1}, m_v^t).$$

Stacking several layers lets information travel farther through the graph: after one layer, a vertex representation depends on its immediate neighbors; after two layers, it can depend on vertices at distance two; and so on. This is why local quantities such as degree are easier for standard GNNs than global quantities such as girth.

Grohe's survey on the logic of graph neural networks provides the theoretical framing for this limitation [6]. Standard message-passing GNNs are closely related to color refinement and the one-dimensional Weisfeiler–Leman algorithm. This connection is important for the thesis because it explains why some graph properties are naturally accessible to ordinary GNNs while other global or symmetry-sensitive properties may require stronger architectures, additional features, or a different formulation of the search problem.

The project uses several GNN architectures:

- GCN, a stable baseline based on degree-normalized neighbor aggregation [7].
- GraphSAGE, an inductive architecture based on learned aggregation [8].
- GIN, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [9].
- GINE, an edge-aware variant of GIN used when edge attributes such as voltage labels are part of the input [10].
- GPS, a graph transformer architecture combining local message passing with global attention [11].
- Loopy GNN, a cycle-aware architecture relevant to minimum-cycle and girth-related tasks [12].

4 Related Work

The thesis combines two lines of work: the graph-theoretic study of cages and the use of neural models as heuristics for discrete search. The purpose of this chapter is not

to survey all cage constructions. Instead, it identifies the methods that matter for the experiments: algebraic constructions that restrict the search space, computational search methods that improve known upper bounds, and graph neural networks that can be used as learned scoring functions.

4.1 Classical and Computational Cage Construction

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision. The Dynamic Cage Survey provides the broad background and record-holder context [1].

For this thesis, the most relevant lesson from classical construction is that successful methods rarely search over arbitrary graphs. They usually impose structure first and search only inside the remaining degrees of freedom. This can mean choosing a highly symmetric graph family, generating graphs from a group action, or constructing a lift of a smaller base graph. The representation then guarantees part of the target property, while the remaining constraints are checked or optimized computationally.

Recent computational work follows this pattern. Exoo et al. describe lift-based generation, pruning of voltage assignments, tabu search, hill climbing, and excision methods for improving upper bounds on cage and related graph-construction problems [4]. Exoo also studies order-decreasing transformations and graph-distance moves that preserve regularity and girth or make excision possible [13]. In this context, excision means starting from a known regular graph, removing a selected local part, and reconnecting the remaining vertices in a way that preserves regularity and avoids short cycles.

These methods are important because they provide the non-learning baselines for the thesis. A learned method should not only outperform unrestricted random edge editing. The stronger question is whether it can improve a search process that already uses graph-theoretic structure, such as a voltage-lift search with exact short-cycle checks.

4.2 Graph Neural Networks and Learned Heuristics

The machine-learning side of the project should be placed against work on graph neural networks and learned heuristics for combinatorial search, not only against graph generation papers. The relevant question is whether a learned model can guide a hard discrete search procedure. Work such as NeuroSAT showed that neural networks can learn useful signals on symbolic search instances even when they do not replace the exact solver [14]. This is a useful analogy for the present project: the learned component need not solve the problem alone, but can replace or improve a branching or scoring heuristic inside a deterministic algorithm.

This framing is a better fit for the present project than treating a neural network as a complete graph generator. Standard message-passing GNNs are well-suited to permutation-equivariant scoring of graph states, but the cage problem also contains exact constraints that should not be left for the model to rediscover from data. Degree regularity can be enforced by construction, and short-cycle violations can often be detected exactly. The learning problem is therefore to choose between many legal or partially legal moves, not to replace the mathematical verification step.

This distinction also determines how experimental claims should be interpreted. If a GNN-guided method succeeds only on small instances where random search or tabu search already succeeds, the result is mainly a proof that the formulation works. Evidence for a useful contribution would require improvement over these structured baselines, better sample efficiency, or better transfer across base graphs, groups, or target parameters.

5 Preparatory GNN Tasks

Before attempting graph generation, the project first evaluated several GNN architectures on two node-level prediction tasks: degree prediction and minimum-cycle prediction. These tasks are not the final goal of the thesis. They serve as controlled probes for the two structural constraints that define a (k, g)-graph.

Degree prediction is local and exactly verifiable. A vertex degree can be recovered from the immediate neighborhood, so this task tests whether the model, data pipeline, and training loop can learn a simple structural quantity. If an architecture fails to predict degree reliably, it is not a good candidate for harder construction tasks without further tuning.

Minimum-cycle prediction is a harder structural task. For each vertex, the target is the length of the shortest cycle containing that vertex, with target 0 for vertices that do not lie on any cycle. Unlike degree, this cannot be determined from immediate neighbors alone. It requires information about paths that leave a vertex and later return to it. This makes the task closer to the girth constraint used in constructing (k, g)-graphs.

5.1 Data Generation

Both tasks use dynamically generated random graphs. Instead of training on a fixed set of graphs, new Erdos–Renyi graphs are sampled during training. This reduces the chance that a model simply memorizes a finite training set and gives a cheaper way to expose it to many small graph structures.

The current implementation generates graph labels directly from exact algorithms. Degree labels are computed as $\deg(v)$ for each vertex. Minimum-cycle labels are computed by searching for the smallest cycle containing each vertex. This keeps the supervision reliable even though the graphs themselves are random.

The input features are intentionally simple. The code supports either a constant one-dimensional feature or a four-dimensional feature vector consisting of a normalized node index, two random real-valued coordinates, and one additional random scalar. These features should not be interpreted as meaningful graph attributes. They mainly give the neural network a vertex-level input tensor while forcing the model to learn from graph structure.

5.2 Architectures Compared

The same family of architectures is used across the preparatory tasks: GCN, GraphSAGE, GIN, GPS, and Loopy GNN. This comparison is useful because the architectures have different inductive biases. GCN normalizes neighbor messages by degree, which can make raw counting harder. GIN and GraphSAGE preserve count-like information more directly. GPS adds global attention, and Loopy adds explicit cycle-aware information.

5.3 Evaluation

The early experiments used a regression objective with mean squared error and evaluated by rounding predictions to the nearest integer. This produces an intuitive exact-node accuracy, but it is a brittle metric: a prediction off by one receives no credit. For this reason, the results are reported using exact rounded accuracy together with mean absolute error, root mean squared error, and the fraction of predictions that are off by one.

The final evaluation battery contains 75 graphs with 1790 vertices in total. It combines random Erdos–Renyi graphs with structured examples such as complete graphs, complete bipartite graphs, cycles, grids, hypercubes, and known regular graphs such as the Petersen and Heawood graphs. Each trained model is evaluated on the same graphs.

5.4 Degree Prediction Results

The degree task behaves as expected. Architectures with sum-like or inductive aggregation learn it essentially perfectly, while degree-normalized GCN variants perform much worse. This does not mean that GCN is useless in general; it means that this particular raw-counting task is poorly matched to normalized aggregation.

Model family	Accuracy	MAE	RMSE
SAGE, small	1.000	0.000	0.000
GIN, small	1.000	0.000	0.000
SAGE, large	1.000	0.000	0.000
GIN, large	0.998	0.002	0.041
GPS-GIN, small	0.998	0.002	0.041
GPS-GIN, large	0.978	0.022	0.148
Best GCN variant	0.434	0.638	0.888
Best Loopy variant	0.401	0.925	2.876

Table 1: Degree prediction on 75 validation graphs with 1790 vertices. Several GIN/ SAGE-based models solve the local counting task, while normalized GCN variants are much weaker.

The result confirms that the training pipeline can learn a simple structural quantity. It also gives a useful warning: stronger or more specialized architectures are not automatically better on every task. Loopy GNN is designed to expose cycle information, but on degree prediction this extra channel does not help the basic counting problem.

5.5 Minimum-Cycle Prediction Results

Minimum-cycle prediction is substantially harder. The target asks for the length of the shortest cycle containing each vertex, with value 0 for vertices not contained in any cycle. This requires information beyond the immediate neighborhood and is closer to the girth constraint that appears in constructing (k, g)-graphs. The larger Loopy model is the clear winner in this setting, which supports the intuition that cycle-aware architecture is useful for cycle-related tasks.

Model family	Accuracy	MAE	RMSE	Off by 1
Loopy, large	0.647	1.287	2.834	0.193
Loopy, small	0.393	1.969	3.162	0.182
GIN, small	0.391	1.346	2.125	0.253
SAGE, large	0.266	1.645	2.432	0.319
GPS-GIN, small	0.253	3.382	4.138	0.000
Best GCN variant	0.203	1.861	2.435	0.259

Table 2: Minimum-cycle prediction on the same 75 validation graphs. The task is much harder than degree prediction, and the best result comes from the larger cycle-aware Loopy model.

The minimum-cycle task is not solved by these models. Even the best result misses a large fraction of vertices. This negative result is useful: it suggests that girth-related information should not be treated as a simple supervised prediction target, and it motivates the later move toward search formulations where exact graph-theoretic checks remain part of the algorithm.

5.6 Lessons From the First Experiments

The first small-capacity experiments were useful, but they should not be treated as final evidence about the limits of the architectures. A reviewer of the earlier SVK paper correctly pointed out that hidden dimension 32 with four layers is an underpowered setting for strong architectural claims. The correct conclusion is narrower: in the initial configuration, GIN and GraphSAGE were the most reliable for degree prediction, while minimum-cycle prediction remained difficult and exposed the need for larger models or more cycle-aware architectures.

Later project documentation indicates that increasing capacity changes the picture, especially for Loopy and GPS. This supports a more cautious thesis framing: the preparatory tasks identify promising model families and failure modes, but they do not by themselves solve the problem of constructing (k, g)-graphs.

The main value of this chapter is therefore methodological. It explains why the project did not jump directly into graph generation. Degree prediction tested basic structural learning, while minimum-cycle prediction tested whether the models could learn information related to girth.

The conclusion is therefore not that one architecture should be used everywhere. The conclusion is that the learning formulation matters. Degree is a local counting

problem; minimum-cycle prediction is a cycle-sensitive structural problem; constructing a (k, g)-graph is harder still because the algorithm must choose actions, not only predict labels.

6 Construction Methods for (k, g)-Graphs

The target problem is to construct small (k, g)-graphs. This is harder than predicting a graph invariant, because the algorithm must actively choose edges while preserving regularity and avoiding short cycles. The search space grows quickly with the number of vertices, and most arbitrary edge choices lead to invalid or unpromising graphs.

The problem is therefore more naturally viewed as a search problem than as a single prediction problem. A construction algorithm repeatedly holds a partial object, chooses a modification, checks which constraints are still satisfiable, and continues until it either reaches a valid graph or enters an unpromising region of the search space. Machine learning can enter this process in several different ways: as a learned heuristic for deterministic search, as a policy trained from complete construction attempts, or as a scoring function inside a constrained algebraic search.

6.1 Guided Search as the First Idea

A natural first idea is to use deterministic search and let a GNN guide it. For example, one can build a graph edge by edge and use a learned scoring function to prioritize partial graphs that appear more likely to extend to a valid (k, g)-graph.

The difficulty is not the search loop itself. The difficulty is obtaining useful labels for partial states. Positive examples can be obtained by taking subgraphs of known valid graphs. Useful negative examples are harder: a partial graph may look bad but still be extendable after adding many more edges. Deciding whether a partial graph can be completed into a valid (k, g)-graph is itself a hard search problem. As a result, the supervised version risks training on easy examples far from the decision boundary, which gives little useful guidance to the search.

This explains why the project moved away from a purely supervised A-star-style approach. The obstacle is not that guided search is unnatural; on the contrary, it is a natural formulation. The obstacle is that the labels needed to train a good heuristic already require solving a difficult completion problem.

6.2 Why Reinforcement Learning Was Tried

Reinforcement learning is one standard way to train a policy for a sequential decision problem when reliable intermediate labels are unavailable. Instead of asking for a dataset that says whether each partial graph is extendable, the model learns from complete construction attempts. The environment supplies rewards based on the observed consequences of actions: whether an edge edit was valid, whether the graph moved closer to regularity, whether short cycles were avoided, and whether the final graph satisfies the target constraints.

In this sense, the reinforcement-learning approach is not separate from search. It is a learned version of heuristic search. A hand-designed search algorithm chooses moves according to a manually specified rule; an RL policy attempts to learn such a rule from repeated interaction with the construction environment. This makes it a reasonable first learned-construction method after the supervised heuristic approach becomes difficult to label.

At the same time, this choice should be interpreted cautiously. Direct reinforcement learning does not build in much graph-theoretic structure. The agent must learn which vertices should become active, which edges should be added or removed, how to satisfy degree constraints, and how to avoid short cycles. The experiment is therefore useful partly as a diagnostic: it tests how far an unconstrained learned search formulation can go before stronger mathematical structure has to be inserted into the representation.

6.3 Direct Reinforcement Learning

The next approach reframed construction as a sequential decision problem and used Proximal Policy Optimization [15]. The environment represents a partially constructed graph. Each action corresponds to an unordered pair of vertices. If the edge is absent, the action attempts to add it; if the edge is present, the action attempts to remove it.

This direct graph-editing formulation has an advantage: it does not require a precomputed dataset of labeled partial graphs. The model learns from interaction. However, it also makes reward design and action pruning essential.

Several invalid or unhelpful actions can be ruled out directly:

- An edge is not added if either endpoint already has degree k.
- An edge is not added if it would create a cycle shorter than g. If adding edge (u, v) creates a new short cycle, that cycle must include the new edge, so it is enough to check the shortest path between u and v before the edge is added.
- An edge is not removed if doing so disconnects the active part of the graph.
- An edge is not removed if too few active vertices would remain to reach the Moore-bound target.

The initial reward was sparse: small rewards for adding edges, penalties for invalid or removing actions, and a large terminal reward for constructing a valid graph. This was not enough. The agent often reached dead-end states and received little directional signal about how to improve.

The later reward added progress-based shaping. A potential function measured partial progress toward the target, combining degree regularity and edge count. The reward then included the change in potential after each action. This made intermediate improvements visible to the agent.

One important lesson was that the usual discounted potential-shaping expression can behave badly in long unfinished episodes. With discount factor $\gamma < 1$, maintaining a high-potential but incomplete graph creates an implicit per-step penalty. The usual policy-invariance result for potential-based shaping assumes the matching discounted form; once the practical environment deviates from this idealized setting, the reward must be treated as an engineering heuristic rather than as a theorem-preserving transformation. In this project, the more practical choice was the undiscounted potential difference $\Phi(s') - \Phi(s)$.

6.4 Curriculum Learning

The environment samples parameter pairs in increasing difficulty, ordered by the Moore bound. Training starts with small cases such as $(3, 5)$ and $(3, 6)$. A new stage is unlocked only after the agent reaches a required success rate over a recent window of episodes. In the experiments reported here, the exact threshold and window were implementation parameters rather than theoretical constants; they should be interpreted as part of the training setup. This curriculum avoids starting immediately on parameter pairs where successful episodes are too rare to provide a useful learning signal.

6.5 Limits of the Direct Formulation

The direct graph-editing formulation is useful for experimentation, but it has structural weaknesses. Regularity is not guaranteed; it must be learned or enforced through action pruning and rewards. The action space also grows quadratically with the number of available vertices: on a graph with n vertices, there are $\binom{n}{2} = \frac{n(n-1)}{2}$ possible unordered pairs, and each pair can become an add-or-remove decision depending on the current graph. Finally, success on small target pairs does not by itself imply that the method scales to harder open cases.

For this reason, results from the direct PPO environment should be reported as exploratory unless they are compared against strong baselines such as random search, greedy search, brute force on small instances, or known construction methods. Its main role in this thesis is methodological: it shows that learning from interaction is possible, but that too much capacity is spent rediscovering constraints that are already known from graph theory.

6.6 Transition to Voltage Lifts

The later direction uses voltage graph lifts. This is not just a different neural-network architecture; it is a different representation of the construction problem. The search starts with a small base graph and a finite group. The algorithm assigns group elements, called voltages, to the base graph edges. The lift then produces a larger graph.

This representation is attractive because regularity is built into the construction: if the base graph is k-regular, the lift is also k-regular. The learning problem can therefore focus more directly on avoiding short cycles and choosing promising voltage assignments.

Thus the transition from direct RL to voltage lifts is not a change from one arbitrary experiment to another. It is a response to the weaknesses of the first formulation. The direct environment made the construction problem as general as possible; the voltage formulation instead uses known algebraic structure to remove regularity from the learning problem and to provide a cheaper exact signal for girth violations.

The current codebase includes exact girth computation for voltage lifts using net-voltage analysis, random search, exhaustive search for small groups, tabu search, GNN-guided beam search, meta-search over base graphs and groups, and a reinforcement-learning environment for voltage assignment. This formulation also aligns better with recent non-AI work on small regular graphs, where pruning, tabu search, hill climbing, and excision are applied inside highly constrained search spaces [4]. The detailed construction of voltage lifts is described in the next chapter.

7 Voltage Graph Lifts

The direct graph-editing environment asks the learning algorithm to satisfy two difficult constraints at the same time: every vertex should have degree k, and the graph should avoid all cycles shorter than g. Voltage graph lifts remove one of these burdens. If the base graph is k-regular, then every lift of that base graph is also k -regular. The search therefore shifts from constructing all edges of a large graph to choosing group labels on the edges of a much smaller base graph.

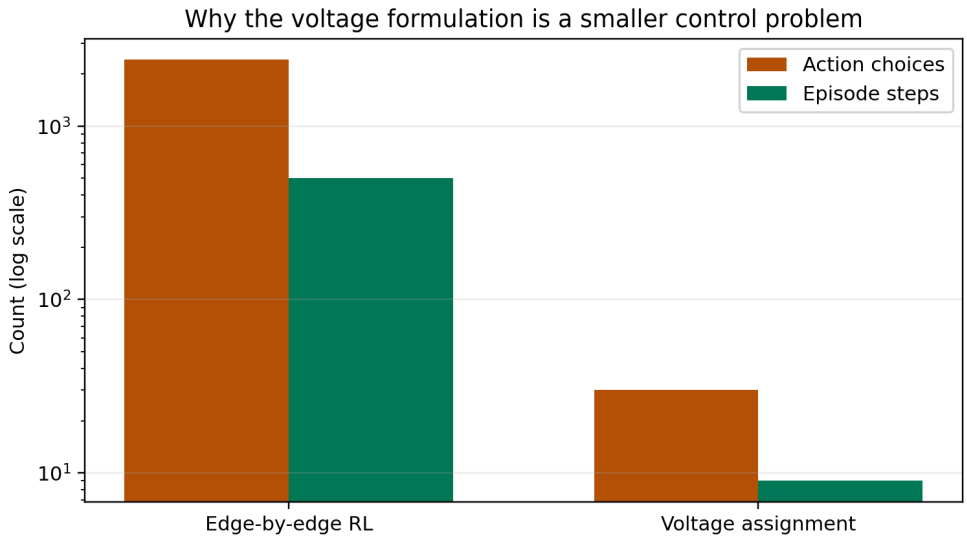

Figure 1: The voltage formulation has a much smaller control problem than direct edge-by-edge graph editing. The numbers shown are illustrative: a 70-vertex direct environment has thousands of possible edge actions, while a voltage assignment step chooses one group element.

7.1 Construction

A voltage-lift construction uses three ingredients:

- a small base graph B,
- a finite group Γ,
- a voltage assignment α that maps every directed edge of B to an element of Γ.

For undirected graphs, each edge is represented by two opposite directed arcs. If one direction has voltage a, then the reverse direction receives a^{-1}. The lift has vertex set $V(B) \times \Gamma$. For an arc from u to v with voltage a, the lift connects

$$(u, h) \quad \text{to} \quad (v, ha)$$

for every group element $h \in \Gamma$. This thesis uses right multiplication by the voltage element. For abelian groups this convention is invisible, but it matters for non-abelian groups because the order of multiplication is part of the construction.

In the codebase, this construction is implemented in `ai/cage/voltage/lift.py`. A vertex pair (u, h) is encoded as the integer `u * group.order + h`. The resulting graph has $|V(B)| \cdot |\Gamma|$ vertices. The verification step checks connectedness, k-regularity, and girth.

The following figure illustrates the construction on the base graph with two vertices and three parallel edges. The group is Z_7, so each base vertex becomes seven vertices in the lift. If a base edge has voltage a, then from each copy (u, h) the lifted edge goes to $(v, h + a)$, where addition is taken modulo 7. The three base edges therefore define three perfect matchings between the two groups of seven lifted

vertices. With voltages 0, 1, and 3, the resulting graph has 14 vertices, is 3-regular, and has girth 6.

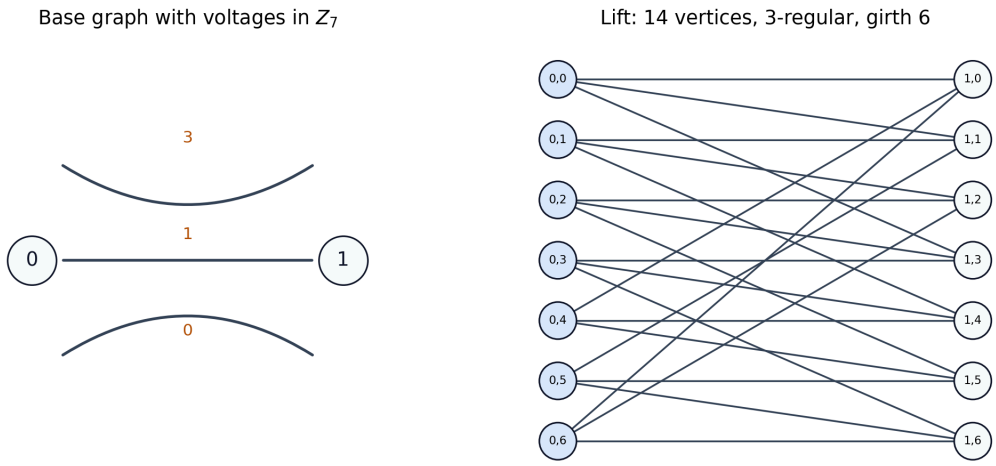

Figure 2: A two-vertex base graph with three parallel edges and voltages in Z_7 lifts to a 14-vertex cubic graph. With a suitable assignment, this produces the Heawood graph, a $(3, 6)$-graph of minimum order.

7.2 Why This Helps

The most important property is preservation of regularity. In direct graph editing, the agent must learn to keep every vertex near degree k. In the voltage formulation, this is structural. A k-regular base graph produces a k-regular lift automatically. This is why the approach is a better match for constructing (k, g)-graphs than unrestricted graph generation.

The second advantage is that girth can be analyzed on the base graph. Consider a closed non-backtracking walk W in the base graph. Multiplying the voltages along the directed arcs of W gives its net voltage. If this product is the identity element of Γ, then following the corresponding lifted edges returns not only to the same base vertex but also to the same group element. Such a walk therefore creates a cycle candidate in the lift. Conversely, every cycle in the lift projects to a closed walk in the base graph with identity net voltage.

This is the reason voltage lifts are useful computationally: short-cycle violations can be detected without constructing the full lifted graph. The implementation in `ai/cage/voltage/cycle_analysis.py` enumerates short closed walks in the base graph and multiplies voltages along the walk.

This gives a cheap exact cost for search:

- if a short closed walk has identity net voltage, it corresponds to a short cycle in the lift;
- if no such walk exists below length g, the lift has girth at least g up to the checked bound.

This exact cost is used later by the search and reinforcement-learning experiments in the experiments chapter. It is also the main reason the learned component can be placed inside a constrained search procedure instead of being responsible for generating an entire graph from scratch.

7.3 Search Methods in the Current Framework

The voltage framework contains several search strategies. They differ in how they choose voltage assignments, but they all keep exact verification at the end: a generated graph is accepted only after connectedness, regularity, and girth are checked.

Method	Search object	Guidance signal	Verification
Exhaustive search	All voltage assignments	None; enumerates all choices	Exact lift check
Random search	Sampled voltage assignments	Uniform random choices	Exact lift check
Tabu search	One-voltage changes	Number of short identity-voltage walks	Exact lift check
GNN-guided beam search	Partial voltage assignments	`GirthPredictor` score	Exact lift check
Meta-search	Base graph and group choices	Enumeration over configured families	Exact lift check
Voltage-assignment PPO	Sequential voltage assignment	Learned policy and reward	Exact lift check

Table 3: Search strategies in the voltage framework. The learned components guide the search, but they do not replace exact verification of the resulting (k, g)-graph.

For small groups and few base edges, exhaustive search enumerates every voltage assignment. This is only practical when $|\Gamma|^m$ is small, where m is the number of undirected base edges. Random search samples assignments and verifies them; it is simple, but it is an important baseline because some small cases are easy enough that random search succeeds.

The tabu search implementation follows recent computational work on small regular graphs [4]. It fixes tree-edge voltages to the identity, so only non-tree edges are searched. It then changes one voltage at a time and minimizes the number of short identity-voltage walks. A tabu list prevents the algorithm from immediately undoing recent moves.

The GNN-guided beam search assigns voltages edge by edge. A trained `GirthPredictor`, defined and evaluated in Chapter 7, scores partial assignments, and the search keeps only the highest-scoring candidates. The `meta_search` function searches over base graphs and finite groups. For cubic graphs, the candidates include the dumbbell base, a four-node cubic base, the prism base, and a Petersen-like base. Candidate groups include cyclic groups, dihedral groups, direct products, and semidirect products.

The reinforcement-learning environment in `ai/cage/voltage/rl_env.py` treats one voltage assignment as one episode. The state is the base graph with partial voltage labels, the action is a group element, and the episode length is the number of base edges. This is much shorter than the direct edge-editing environment. The model in `ai/cage/voltage/rl_model.py` uses either a GINE-only encoder or a GPS encoder with GINE as the local message-passing component, followed by global pooling, an actor head over group elements, and a critic head for PPO.

7.4 Current Interpretation

The important thesis claim should be modest. Voltage lifts are not new, and many hand-crafted or computational lift searches already exist. The contribution of this project should not be described as inventing voltage lifts. The relevant question is whether graph neural networks can guide this constrained search space better than simple baselines.

This framing also makes negative results more useful. If the learned component does not outperform random search, tabu search, or beam search, that is still informative: it shows that the chosen learned representation did not capture enough reusable structure. If it does outperform them on some configurations, the result is stronger because the comparison is against methods that already respect the mathematics of the problem.

8 Experiments and Results

This chapter reports the experiments in the same order as the method developed. The purpose is not to present a single isolated number, but to explain what each experiment showed and why the next formulation was needed.

The preparatory experiments in Chapter 4 show that GNNs can learn local graph structure, but that cycle-sensitive quantities are harder. This chapter starts from the later experiments: predicting the quality of voltage assignments, training a voltage-assignment policy, and comparing complete methods for constructing (k, g)-graphs.

8.1 Voltage-Girth Predictor

The voltage-lift experiments use a `GirthPredictor` to score voltage-labeled base graphs. The input is a small directed base graph with node features and edge attributes encoding voltage labels. The model embeds the voltage labels, applies

several GINE layers, pools the graph representation, concatenates global context features $(k, g, |\Gamma|)$, and predicts both the girth and the binary event girth $\geq g$.

Three training regimes were evaluated. Variant A trains a separate model for one target pair, here the cubic targets $(3, g)$. Variant B trains one model for a fixed girth value and several degrees. Variant C trains one unified model over all considered targets. This naming is used only to make the comparison readable; the important distinction is how much parameter sharing is forced across targets.

Variant	$g = 5$	$g = 6$	$g = 7$	$g = 8$	$g = 9$	$g = 10$
A: cubic only	0.828	0.786	0.642	0.537	0.450	0.385
B: fixed girth	0.726	0.700	0.640	0.525	0.421	0.391
C: unified, cubic slice	0.823	0.800	0.659	0.524	0.488	0.370

Table 4: F1 scores for voltage-girth prediction on cubic targets. Performance decreases as the target girth increases and positive examples become rarer.

The unified model has overall F1 score 0.694, but this aggregate number hides important variation between targets. It performs well on easier or denser-positive targets such as $(3, 5)$, $(3, 6)$, $(4, 5)$, and $(4, 6)$, but it weakens sharply on sparse targets such as $(4, 7)$ and $(4, 8)$. The target $(5, 7)$ has no positive examples in the generated dataset, so its F1 score is 0 even though accuracy is 1.000; this is exactly why accuracy alone is misleading for this task.

Target	F1	Target	F1	Target	F1
$(3, 5)$	0.823	$(4, 5)$	0.779	$(5, 5)$	0.613
$(3, 6)$	0.800	$(4, 6)$	0.800	$(5, 6)$	0.613
$(3, 7)$	0.659	$(4, 7)$	0.147	$(5, 7)$	0.000
$(3, 8)$	0.524	$(4, 8)$	0.248		
$(3, 9)$	0.488				
$(3, 10)$	0.370				

Table 5: Per-target F1 scores for the unified voltage-girth predictor. The aggregate score is not enough; the hard sparse-positive targets are visibly weaker.

The conclusion from these predictor experiments is mixed but useful. The model learns a nontrivial signal about voltage assignments, especially for smaller targets, but it is not uniformly reliable across the search space. This suggests that it should be used as a heuristic inside verified search rather than as a replacement for exact girth checking.

8.2 Voltage-Assignment Reinforcement Learning

The voltage formulation was also used as a reinforcement-learning environment. Each episode assigns voltages to the base graph edges. This is much shorter than direct edge editing: the policy chooses group elements for a small base graph rather than choosing among all possible edges of a large graph.

Two GPS-based voltage policies were trained. Both used three message-passing layers and PPO. The smaller model used hidden dimension 64 and four attention heads; the larger model used hidden dimension 128 and eight heads. Both runs reached stage 6 of the curriculum after 6.5 million environment steps.

Voltage PPO model	Hidden dim.	Heads	Final steps	Final stage
Compact GPS policy	64	4	6.5M	6
Larger GPS policy	128	8	6.5M	6

Table 6: Voltage-assignment PPO training progress. Both policies unlocked six curriculum stages, showing that the voltage environment is learnable, but not yet proving superiority over non-learned search.

The stage-unlock histories show a useful difference between the two runs. The larger model reached intermediate stages faster: it unlocked stage 4 after roughly 32 thousand steps and stage 5 after roughly 469 thousand steps, while the compact model reached the same stages after roughly 197 thousand and 1.30 million steps. However, both runs required several million steps to reach stage 6, and the best average reward became worse on later stages. This indicates that the curriculum becomes substantially harder and that reaching early stages is not enough evidence of scalable construction of (k, g)-graphs.

8.3 Generator Comparison

The central thesis question is whether learning improves search for (k, g)-graphs. The experiments above show that the learned components are meaningful: GNNs can learn relevant graph properties, the voltage-girth predictor captures a real but uneven signal, and PPO can learn in the voltage-assignment environment. They do not by themselves prove that a learned generator is better than strong non-learned baselines.

The final validation run is stored in `validation_runs/2026-05-04_18-21-59`. It evaluates 13 target pairs, from $(3, 5)$ to $(5, 7)$, with two random seeds per method. The nominal budget was one minute per trial. In a few timeout cases the asynchro-

nous backend returned later than the nominal budget, so wall-clock time should be interpreted cautiously; success counts are the primary metric.

Method	Success	Avg. time	Avg. steps	Avg. order	Role
Random edge editing	10/26	10.0 s	5049	24.4	baseline
Heuristic search	12/26	3.7 s	699	24.3	baseline
Direct PPO	2/26	21.2 s	1370	12.0	learned
Voltage search	19/26	5.8 s	45	53.9	structured
Voltage + unified predictor	20/26	8.7 s	57	52.5	learned
Voltage PPO	0/52	-	-	-	learned

Table 7: Aggregate generator comparison over the full validation run. Averages are computed only over successful trials. Step counts are not directly comparable across method families, because they count different operations.

The most important result is that the voltage representation changes the difficulty of the problem. Direct random search and heuristic search solve some small targets, but they fail more often as g or k increases. Direct PPO succeeds only on $(3, 5)$. In contrast, voltage search succeeds on 19 of 26 trials, and voltage search guided by the unified predictor succeeds on 20 of 26 trials. This supports the main structural claim of the thesis: building regularity into the representation is more valuable than asking a neural policy to rediscover it from edge edits.

Target	Rand.	Heur.	PPO	Volt.	Volt. +U	Volt. +S	Volt. PPO
$(3, 5)$	2/2	2/2	2/2	2/2	2/2	2/2	0/4
$(3, 6)$	2/2	2/2	0/2	2/2	2/2	2/2	0/4
$(3, 7)$	0/2	2/2	0/2	2/2	2/2	2/2	0/4
$(3, 8)$	2/2	2/2	0/2	2/2	2/2	2/2	0/4
$(3, 9)$	0/2	0/2	0/2	1/2	2/2	1/2	0/4
$(3, 10)$	0/2	0/2	0/2	2/2	2/2	2/2	0/4
$(4, 5)$	0/2	0/2	0/2	2/2	2/2	2/2	0/4
$(4, 6)$	2/2	2/2	0/2	2/2	2/2	2/2	0/4
$(4, 7)$	0/2	0/2	0/2	0/2	0/2	0/2	0/4
$(4, 8)$	0/2	0/2	0/2	0/2	0/2	0/2	0/4
$(5, 5)$	0/2	0/2	0/2	2/2	2/2	2/2	0/4
$(5, 6)$	2/2	2/2	0/2	2/2	2/2	2/2	0/4
$(5, 7)$	0/2	0/2	0/2	0/2	0/2	0/2	0/4

Table 8: Success counts by target in the full validation run. `Volt.+U` uses the unified GirthPredictor, while `Volt.+S` uses the matching per-girth specialist predictor when available.

The learned girth predictor gives a small improvement in this validation, but not a decisive one. The unified predictor succeeds once more than the non-learned voltage baseline, mainly on $(3, 9)$, where plain voltage search succeeds in one of two seeds and the unified predictor succeeds in both. On many easier targets, both methods already succeed, so there is little room to show an improvement. On the hard targets $(4, 7)$, $(4, 8)$, and $(5, 7)$, none of the tested methods succeeds.

The voltage PPO result is negative in this validation: both trained voltage policies fail on all tested targets. This is surprising because the training metadata shows that both policies reached stage 6 of the curriculum. The current validation records contain no successful final graph for these runs. [Check whether this is a genuine policy failure or a generator/API integration problem before making the final claim stronger.]

Overall, the generator comparison supports a restrained conclusion. The project does not show that machine learning is already better than structured non-learned search. It shows that the algebraic voltage formulation is much more effective than direct edge editing, and that learned components can be inserted into this formulation, but current learned guidance is only competitive with, not clearly superior to, the non-learned voltage baseline.

9 Future Work

The experiments in this thesis explore several ways of applying machine learning to the generation of graphs with prescribed degree and girth. The first supervised idea was to train a model to recognize promising partial graphs and use it as a heuristic inside a search procedure. This direction is natural, but it is difficult to obtain reliable labels: deciding whether a partial graph is close to a valid completion can itself require solving a hard construction problem.

Direct reinforcement learning avoids these labels by learning from complete construction attempts. It showed that a GNN policy can be trained in this setting, but it also exposed a weakness of an unconstrained graph-editing formulation. The agent has to learn regularity, avoid short cycles, choose useful vertices, and recover from bad intermediate states at the same time. A large part of the learning problem is therefore spent rediscovering constraints that are already known mathematically.

The most promising direction is the voltage-lift formulation. It inserts graph theory into the representation before learning begins: regularity is guaranteed by the base graph, and short cycles can be detected through identity-voltage walks in the base graph. This suggests that future progress is more likely to come from combining learned components with constrained algebraic search than from unrestricted neural graph generation.

9.1 Stronger Voltage-Lift Search

The current voltage framework already includes random search, tabu search, GNN-guided beam search, and reinforcement learning over voltage assignments. The most direct improvement is better pruning before learning is applied. Recent computational work on small regular graphs uses constraints from short closed walks, spanning-tree normalization, canonicalization under automorphisms, and careful rejection of equivalent assignments [4]. A stronger pruning layer would reduce the search space before a GNN is asked to score candidates.

9.2 Better Data and Training Signals

The learned model could be trained to predict more than the final girth. The short-walk cost used by tabu search is a dense signal: it counts how many short closed walks currently have identity net voltage. This quantity could be used as an auxiliary target, as a reward-shaping term, or as part of the beam-search score. Such a model would not only predict whether a final assignment is good, but also recognize why a partial assignment is becoming dangerous.

The data should also be improved on hard targets where positive examples are rare. More balanced sampling near the decision boundary, more diverse base graphs and groups, and explicit hard-negative mining could make the learned scorer sturdier than the current model.

9.3 Learned Move Ranking

The GNN need not generate the whole voltage assignment by itself. A more robust role may be to rank moves inside an existing search algorithm. For example, a model could score candidate voltage changes in tabu search, select which base-graph and group pairs deserve more compute, or choose which partial assignments should survive in a beam. Exact verification would remain unchanged; the learned component would only influence which parts of the search space are explored first.

9.4 Excision and Completion

Another promising direction is excision. Instead of constructing a graph from scratch, one can start with a known graph, remove selected vertices or local structures, and reconnect the remaining graph while preserving regularity and girth. This family of methods appears in recent computational work on decreasing the order of regular graphs [4], [13].

Machine learning could enter this setting as a heuristic for choosing which vertices to remove or which reconnection patterns to try first. Compared with unrestricted edge-by-edge construction, this would again keep the search close to a mathematically meaningful space: the input graph is already near the target class, and the algorithm only has to repair a constrained local defect.

9.5 Toward the Original Goal

The long-term goal is to generate new record graphs or to learn useful algebraic graph properties. The work so far suggests a concrete route toward that goal: use graph theory to define a constrained search space, use exact algorithms to verify every candidate, and use machine learning only where it can improve prioritization inside that search. Voltage lifts currently provide the clearest version of this strategy. Excision and learned move selection are natural next steps if the voltage framework stops improving.

10 Conclusion

This thesis investigated whether graph neural networks can help generate graphs of prescribed degree and girth. The work began with preparatory prediction tasks, continued with direct reinforcement learning over graph edits, and then moved to voltage graph lifts as a more constrained algebraic representation of the construction problem.

The preparatory experiments showed a clear separation between local and cycle-related graph properties. Degree prediction was solved almost perfectly by several GIN- and SAGE-based architectures. Minimum-cycle prediction was much harder, and the best result came from a larger cycle-aware Loopy model. This supports the main methodological lesson: GNNs can learn useful graph structure, but girth-related information should not be treated as an easy generic prediction task.

Direct reinforcement learning was a natural first construction method because the problem is sequential and reliable labels for partial graphs are hard to obtain. However, the direct graph-editing formulation forced the agent to learn too many known constraints at once: regularity, connectivity, edge validity, and avoidance of short cycles. This made it useful as an exploratory baseline, but not as the most promising final formulation.

The voltage-lift framework is the strongest direction developed in the thesis. It builds regularity into the representation and allows short-cycle violations to be evaluated through net voltages on the base graph. The trained voltage-girth predictor learned a nontrivial but uneven signal, and voltage-assignment PPO was able to progress through several curriculum stages.

The generator comparison confirms that this change of representation is the main positive result. Direct reinforcement learning over edge edits solved only the smallest tested target pair, while voltage search solved most of the tested targets. Adding the unified learned girth predictor slightly improved the success count over plain voltage search, but the difference was small. The current experiments therefore do not show that machine learning clearly outperforms structured non-learned voltage search.

The voltage-assignment PPO result is negative in the current validation: neither trained voltage policy produced a valid graph in the tested runs. [Check whether this should be described as a policy failure or as an implementation issue in the validation/generator wrapper.]

The main conclusion should remain cautious. Machine learning appears most useful here as a heuristic inside constrained graph-theoretic search, not as an unrestricted graph generator. The clearest outcome of the work is the movement from unrestricted edge editing toward mathematically structured search spaces.

Bibliography

[1] G. Exoo and R. Jajcay, "Dynamic Cage Survey," *The Electronic Journal of Combinatorics*, 2013.

[2] R. M. Tanner, "A Recursive Approach to Low Complexity Codes," *IEEE Transactions on Information Theory*, vol. 27, no. 5, pp. 533–547, 1981, doi: 10.1109/TIT.1981.1056404.

[3] M. Besta and T. Hoefler, "Slim Fly: A Cost Effective Low-Diameter Network Topology," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014.

[4] G. Exoo, J. Goedgebeur, J. Jooken, L. Stubbe, and T. Van den Eede, "New small regular graphs of given girth: the cage problem and beyond." 2025.

[5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[6] M. Grohe, "The Logic of Graph Neural Networks." 2022.

[7] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, 2017.

[9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *International Conference on Learning Representations*, 2019.

[10] W. Hu *et al.*, "Strategies for Pre-training Graph Neural Networks," in *International Conference on Learning Representations*, 2020.

[11] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.

[12] R. Paolino, C. Vignac, and A. Loukas, "Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning," in *International Conference on Learning Representations*, 2024.

[13] G. Exoo, "On decreasing the orders of (k, g)-graphs", *Journal of Combinatorial Optimization*, 2023, doi: 10.1007/s10878-023-01092-9.

[14] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, "Learning a SAT Solver from Single-Bit Supervision," in *International Conference on Learning Representations*, 2019.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.